Artifact Abstract

On the Reliability of Code Comprehension Proxies

Paper title

On the Reliability of Code Comprehension Proxies

Link to the accepted paper

https://arxiv.org/abs/2605.23008

Purpose

This artifact contains the data, scripts, web application, and study materials used in the paper. It supports full reproduction of the paper's quantitative results: the correlation analyses between student comprehension proxies (accuracy, timing, self-reported difficulty) and expert-ranked ground truth (Figures 3–11), the task taxonomy from open coding (Figure 1, Table 1), the impact of participant background and fatigue on proxy reliability (Table 7), cross-institution consistency between University 1 and University 2 (Table 8), and the evolution of expert agreement across the three Delphi consensus rounds (Table 4).

Badge

We are applying for the **Artifacts Available** and **Artifacts Reusable** badges.

- **Available**: the artifact is permanently archived on Zenodo with a DOI, independent of the authors' personal infrastructure or the GitHub repository.
- **Reusable**: the artifact is self-contained and reproducible end-to-end via Docker with a single command (`python run_all.py` inside the analysis container), completing in under 5 minutes. It includes all raw data, all analysis scripts, the web application used to run the study, all study materials in both machine-readable and human-readable form, and inter-rater agreement scripts. Full justification is in the STATUS file.

Technology skills assumed by the reviewer, and hardware requirements

Reviewers need only basic familiarity with Docker (`docker load` / `docker run`) to reproduce all quantitative results; no Python, Java, or Maven knowledge is required for that path. Building from source (optional) additionally assumes basic Python and Java/Maven familiarity. No special hardware is required — a standard laptop or desktop suffices. Docker images are provided as native builds for both x86_64 and ARM64 (Apple Silicon); no emulation is required on either platform. Full details are in the REQUIREMENTS file.

Provenance

The artifact is archived on Zenodo: https://zenodo.org/records/19348389 (this version's record). The concept DOI **10.5281/zenodo.19348388** represents all versions and always resolves to the latest one. The artifact is also mirrored (non-archival, convenience only) on GitHub at `github.com/erfan-arvan/on-the-reliability-of-code-comprehension-proxies-replication`.

Instructions

1. Install Docker Desktop (includes Docker Compose v2). No other software required.
2. Download `artifact.zip` from the Zenodo record, extract it, and load the Docker image pair matching your architecture (check with `uname -m`; both x86_64 and ARM64 are provided as native builds, no emulation

needed):

```
unzip artifact.zip
cd artifact
# x86_64:
docker load < code-comprehension-analysis-amd64.tar.gz
docker load < code-comprehension-web-amd64.tar.gz
# ARM64 (Apple Silicon):
docker load < code-comprehension-analysis-arm64.tar.gz
docker load < code-comprehension-web-arm64.tar.gz
```

3. Smoke test (expected output: `OK`):

```
docker run --rm code-comprehension-analysis:latest python -c "import pandas,
scipy, matplotlib; print('OK')"
```

4. Reproduce all quantitative results (Figure 1, Tables 1, 3, 4, 6, Figures 3–11, Tables 7–8) in under 5 minutes:

```
mkdir -p results plots
docker run --rm \
  -v "$(pwd)/results:/analysis/results" \
  -v "$(pwd)/plots:/analysis/plots" \
  code-comprehension-analysis:latest python run_all.py
```

   Generated CSVs and plots land directly in `results/` and `plots/` on the host; `--rm` means this can be re-run any number of times with no cleanup needed.
5. (Optional) Run the web application used to conduct the study:

```
docker run --rm -p 8080:8080 code-comprehension-web:latest
```

   then open `http://localhost:8080/admin-login.html` (see `App/README.md`, inside `artifact.zip`, for demo credentials).

Full step-by-step instructions, the dataset schema, and the artifact layout are in `README.MD`, included both standalone in this Zenodo record and inside `artifact.zip`.